

Rapport Final — Projet ETL Data Warehouse

Date : 28 mai 2026

Auteur : Nyanu Kodzo

Environnement : Linux / Docker Compose

Table des matières

1. [Vue d'ensemble du projet](#)
 2. [Planification, jalons et écarts](#)
 3. [Défis rencontrés et solutions](#)
 4. [Architecture technique](#)
 5. [Sources de données](#)
 6. [Couche Bronze — Ingestion avec n8n](#)
 7. [Couche Silver — Nettoyage avec Apache Hop](#)
 8. [Couche Gold — Modélisation dimensionnelle](#)
 9. [Data Warehouse PostgreSQL — Schéma & Modèle en étoile](#)
 10. [Gestion des utilisateurs et accès à la base de données](#)
 11. [Orchestration et planification](#)
 12. [Enrichissement des données — Google Distance Matrix](#)
 13. [Business Intelligence — Metabase & Dashboards](#)
 14. [Infrastructure Docker](#)
 15. [Connectivité réseau Windows ↔ Linux](#)
 16. [Bilan, résultats et guide de démonstration](#)
-

1. Vue d'ensemble du projet

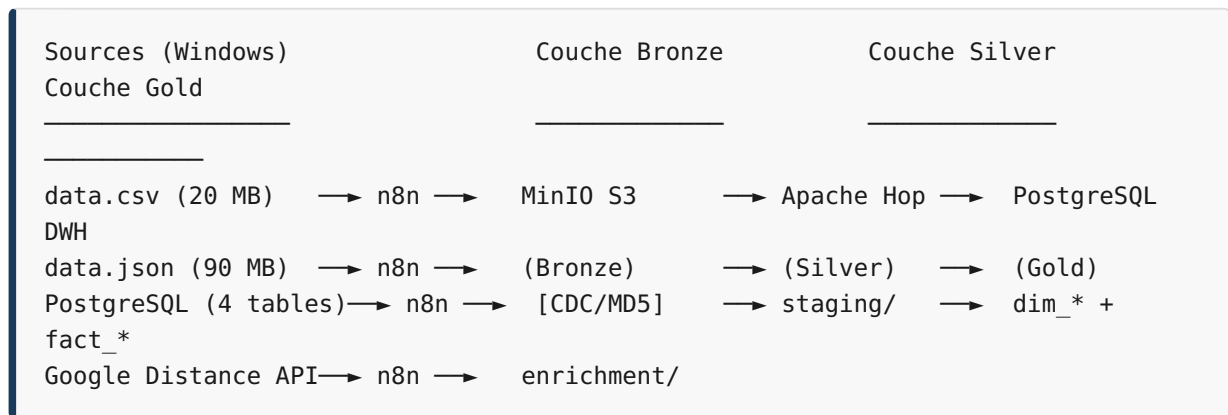
Ce projet consiste en la mise en place d'un pipeline ETL (Extract, Transform, Load) complet et automatisé, suivant une architecture **Medallion (Bronze / Silver / Gold)** afin d'alimenter un Data Warehouse relationnel à des fins d'analyse décisionnelle.

Objectif

Centraliser des données hétérogènes provenant de plusieurs sources (fichiers plats, base de données relationnelle, API externe) dans un entrepôt de données structuré

selon un **modèle en étoile**, accessible via un outil de Business Intelligence permettant une exploration dynamique des données.

Flux global des données



Outils utilisés

Catégorie	Outil / Technologie	Version	Rôle
Orchestration	n8n	2.21.7+	Ingestion + scheduling
Data Lake	MinIO	latest	Stockage Bronze S3
Transformation ETL	Apache Hop	hop-web:latest	Silver + Gold
Data Warehouse	PostgreSQL	16	Stockage structuré
Business Intelligence	Metabase	v0.52.9	Dashboards analytiques
Conteneurisation	Docker / Docker Compose	—	Infrastructure
CDC	MD5 (Node.js <code>crypto</code>)	—	Détection de changements
API Enrichissement	Google Distance Matrix API	—	Distances/durées
Connectivité réseau	Module <code>pg</code> (Node.js)	—	Connexion directe PostgreSQL Windows

2. Planification, jalons et écarts

2.1 Tâches prévues vs tâches réelles

#	Tâche prévue	Durée prévue	Durée réelle	Statut	Écart
1	Mise en place de l'infrastructure Docker (docker-compose, réseau, volumes)	0.5 j	1 j	✓ Fait	+0.5 j — problèmes de compatibilité réseau inter-conteneurs
2	Collecte source 1 : fichiers plats CSV/JSON (ingestion + CDC)	1 j	1.5 j	✓ Fait	+0.5 j — gestion des fichiers binaires volumineux dans n8n
3	Collecte source 2 : PostgreSQL Windows (connexion directe)	1 j	1.5 j	✓ Fait	+0.5 j — configuration du module <code>pg</code> et autorisation <code>NODE_FUNCTION_ALLOW_EXTERNAL</code>
4	Collecte source 3 : API Google Distance Matrix (CDC + déduplication)	0.5 j	1 j	✓ Fait	+0.5 j — gestion des limites de l'API et déduplication des paires
5	Couche Silver : 6 pipelines Apache Hop	1 j	1 j	✓ Fait	0 — conforme au plan
6	Couche Gold : modélisation en étoile (7 pipelines)	1 j	1.5 j	✓ Fait	+0.5 j — corrections FK et typage des données
7	Schéma PostgreSQL Gold (DDL)	0.5 j	0.5 j	✓ Fait	0 — conforme au plan
8	Gestion des utilisateurs BD (admin, vente)	0.5 j	0.5 j	✓ Fait	0 — conforme au plan
9		1 j	2 j	✓ Fait	+1 j — intégration n8n ↔ Apache Hop complexe

#	Tâche prévue	Durée prévue	Durée réelle	Statut	Écart
	Orchestrateur n8n (WF-05 + ETL Runner)				
10	Connexion Metabase au DWH	0.5 j	0.5 j	✅ Fait	0 — conforme au plan
11	Création des dashboards Metabase	1 j	1 j	✅ Fait	0 — conforme au plan
12	Documentation technique	1 j	1 j	✅ Fait	0 — conforme au plan
Total		9.5 j	12.5 j		+3 j

2.2 Jalons du projet

Semaine 1	Semaine 2	Semaine 3
[J1-J2]	[J3-J5]	[J6-J8]
Infrastructure Docker + MinIO + n8n	Sources 1,2,3 ingérées dans MinIO Bronze	Silver+Gold chargé dans PostgreSQL DWH
	[JALON 1]	[JALON 2]
	Données brutes dans le lac	Data Warehouse alimenté
		[JALON 3]
		Dashboards opérationnels

Jalon	Description	Date cible	Date réelle	Écart
J1	Infrastructure Docker 100% opérationnelle	J+2	J+3	+1 j
J2	3 sources ingérées dans MinIO Bronze avec CDC	J+5	J+7	+2 j
J3	Data Warehouse Gold alimenté (dims + faits)	J+8	J+9.5	+1.5 j
J4	Dashboards Metabase livrés	J+9.5	J+10.5	+1 j
J5	Documentation complète + rapport	J+10.5	J+13	+2.5 j

2.3 Analyse des écarts

Les principaux écarts par rapport au planning initial proviennent de :

1. **Connexion directe PostgreSQL Windows** (+0.5 j) : configuration du module `pg` dans n8n (variable `NODE_FUNCTION_ALLOW_EXTERNAL`), validation de l'accessibilité de `10.14.219.2:5433` depuis les conteneurs.
2. **Complexité de l'intégration n8n ↔ Apache Hop** (+1 j) : Apache Hop n'expose pas nativement d'API REST, ce qui a nécessité le développement d'un microservice intermédiaire (`etl_runner`).
3. **Gestion des fichiers binaires dans n8n** (+0.5 j) : les nœuds S3 de n8n effacent les données binaires entre les étapes, nécessitant des nœuds de restauration dédiés.

3. Défis rencontrés et solutions

Défi 1 — Connectivité réseau entre machines hétérogènes

Problème : La machine Windows hébergeant la base PostgreSQL source (`10.14.219.2`) et la machine Linux hébergeant le stack ETL (`10.14.210.159`) appartiennent à des sous-réseaux différents. L'objectif était de permettre aux conteneurs Docker d'atteindre directement la base PostgreSQL Windows.

Solution retenue : Connexion **directe** depuis n8n vers PostgreSQL Windows via l'IP `10.14.219.2` et le port `5433`, grâce au module Node.js `pg`. Les conteneurs Docker ont accès à cette IP via le réseau hôte. Aucun intermédiaire (tunnel, proxy) n'est nécessaire.

```
// Dans WF-03 – connexion directe PostgreSQL Windows
const client = new Client({
  host      : '10.14.219.2',
  port      : 5433,
  database: 'database_extern_local',
  user      : 'postgres',
  password: 'postgres'
});
```

Défi 2 — Perte des données binaires dans n8n entre nœuds S3

Problème : Dans n8n, les nœuds **S3 Upload** effacent systématiquement les données binaires (`binary.data`) de l'item courant après leur utilisation. Lorsqu'un second

upload S3 était nécessaire (version horodatée + `latest/`), la donnée binaire n'était plus disponible.

Solution : Ajout de nœuds **Code intermédiaires** ("Restaurer binaire") qui sauvegardent le contenu binaire dans un champ JSON temporaire avant le premier upload, puis le restaurent avant le second.

```
// Sauvegarder avant upload S3
items[0].json._binaryBackup = items[0].binary.data.data;

// Restaurer après upload S3
items[0].binary = { data: { data: items[0].json._binaryBackup, ... } };
```

Défi 3 — Autoriser l'accès au système de fichiers et à PostgreSQL depuis n8n

Problème : Par défaut, les Code Nodes n8n ne peuvent pas importer de modules Node.js natifs (`fs`, `crypto`) ni de modules externes (`pg`). La lecture des fichiers CSV/JSON depuis le système de fichiers hôte et la connexion directe à PostgreSQL étaient bloquées.

Solution : Configuration de deux variables d'environnement dans le `docker-compose.yml` :

```
NODE_FUNCTION_ALLOW_BUILTIN: "fs,path,crypto,os,http,https,net"
NODE_FUNCTION_ALLOW_EXTERNAL: "pg"
```

Défi 4 — Déclenchement d'Apache Hop depuis n8n

Problème : Apache Hop (outil de transformation ETL) ne dispose pas d'API REST native pour déclencher un workflow à distance. n8n ne peut pas appeler directement `hop-run.sh` à l'intérieur du conteneur `transform_hop`.

Solution : Développement d'un microservice Flask (`etl_runner`) déployé dans un conteneur dédié. Ce service expose un endpoint `POST /run-etl` qui exécute `hop-run.sh` via `docker exec` avec un timeout de 2 heures.

```
n8n → HTTP POST /run-etl → etl_runner → docker exec transform_hop hop-run.sh
```

Défi 5 — CDC sur des fichiers de grande taille (90 MB)

Problème : Le fichier `data.json` pesant 90 MB, le calcul du hash MD5 et le chargement en mémoire par `n8n` ralentissaient considérablement le workflow et pouvaient provoquer des timeouts.

Solution : Calcul du MD5 sur le contenu brut via `fs.readFileSync()` combiné à `crypto.createHash('md5')` avant tout traitement, et comparaison immédiate avec le hash précédent. En cas d'absence de changement, le workflow s'arrête immédiatement sans charger les données en mémoire.

Défi 6 — Typage et mapping des données Silver → Gold

Problème : Les données extraites de la base PostgreSQL Windows et des fichiers JSON présentaient des incohérences de types (dates en chaîne de caractères, prix en string, valeurs nulles implicites), provoquant des erreurs d'insertion dans les tables Gold avec clés étrangères strictes.

Solution : Ajout de transformations explicites dans les pipelines Apache Hop Silver : cast des dates via `TO_DATE()`, conversion des montants via `TO_NUMBER()`, remplacement des nulls par des valeurs par défaut avant insertion.

4. Architecture technique

Composants déployés

Composant	Rôle	Conteneur	IP interne	Port
n8n	Orchestration + Ingestion	<code>ingestion_n8n</code>	172.18.0.5	5678
MinIO	Data Lake Bronze (stockage S3)	<code>datalake_minio</code>	172.18.0.3	9000
Apache Hop	Transformation Silver & Gold	<code>transform_hop</code>	172.18.0.2	8080
PostgreSQL	Data Warehouse Gold	<code>dwh_postgres</code>	172.18.0.4	5432
Metabase	Visualisation BI	<code>analytics_metabase</code>	172.18.0.7	3000
ETL Runner	Déclencheur HTTP → Apache Hop	(microservice Flask)	172.18.0.6	9999

Tous les services sont orchestrés via **Docker Compose** sur un réseau bridge dédié `etl_network` (172.18.0.0/16).

Volumes partagés

- `./staging` → monté dans n8n ET Apache Hop pour un échange de données sans copie réseau.
- `./data_sources` → fichiers sources CSV/JSON accessibles depuis n8n.
- `./minio/data` → stockage persistant des objets S3.
- `./postgres_gold/data` → répertoire de données PostgreSQL persistant.

5. Sources de données

Trois types de sources ont été intégrés, couvrant les trois catégories principales de données :

5.1 Fichiers plats (Flat Files)

Fichier	Taille	Lignes	Format
<code>data.csv</code>	20 MB	~241 912	CSV
<code>data.json</code>	90 MB	~3 000 000	JSON

Schéma : `InvoiceNo`, `StockCode`, `Description`, `Quantity`, `InvoiceDate`, `UnitPrice`, `CustomerID`, `Country`

Ces fichiers contiennent des transactions e-commerce issues d'un système de vente en ligne, fournis depuis une machine Windows via un montage de répertoire partagé (`./data_sources`).

5.2 Base de données PostgreSQL relationnelle (Windows)

Une base source hébergée sur une machine Windows distante (`10.14.219.2:5433`), base `database_extern_local` expose 4 tables relationnelles :

Table	Contenu	Lignes (~)
<code>clients</code>	Données clients (nom, localisation, devise, vendeur)	~300
<code>articles</code>	Catalogue articles (description, prix, coût, catégorie)	~500
<code>salessshipmentheader</code>	En-têtes d'expéditions (dates, destinations, méthode)	~400
<code>salessshipmentline</code>	Lignes d'expéditions (quantités, poids, volumes)	~1 000

5.3 API Google Distance Matrix (enrichissement)

Enrichissement géographique : à partir des couples origine/destination extraits des en-têtes d'expédition, l'API Google Distance Matrix retourne la **distance en km** et la **durée de trajet en minutes** pour chaque itinéraire unique.

6. Couche Bronze — Ingestion avec n8n

La couche Bronze correspond au **stockage brut et versionné** des données dans MinIO (plateforme de stockage objet S3-compatible, non structurée). L'ingestion est implémentée via **5 workflows n8n**.

6.1 Mécanisme CDC (Change Data Capture)

Tous les workflows appliquent un mécanisme de détection de changement basé sur le **hachage MD5** :

1. Calcul du hash MD5 du contenu source.
2. Comparaison avec le hash précédent stocké dans `/staging/_metadata/checksums/{source}.json`.
3. Si le hash a changé → upload dans MinIO (chemin versionné + chemin `latest/`).
4. Si identique → skip immédiat (zéro chargement redondant).

Structure MinIO Bronze :

```
bronze/
├─ raw/flat_files/data_csv/
│  └─ 2026/05/28/20260528_0600/data.csv ← version horodatée
│     └─ latest/data.csv                ← accès rapide dernière version
├─ raw/flat_files/data_json/...
├─ raw/database/clients/...
├─ raw/database/articles/...
├─ raw/database/salesshipmentheader/...
├─ raw/database/salesshipmentline/...
└─ raw/distances/...
```

6.2 Workflows n8n

WF-01 — Ingestion data.csv (CDC)

- Lit `/data_sources/data.csv` via `fs.readFileSync()`
- Calcule le MD5 et vérifie la modification
- Upload versionné + latest dans MinIO Bronze
- Écrit en staging `/staging/flat_files/data.csv`

- Met à jour le fichier de checksum

WF-02 — Ingestion data.json (CDC)

- Même logique que WF-01 appliquée à `data.json`
- Staging : `/staging/flat_files/data.json`

WF-03 — Ingestion PostgreSQL directe (toutes les 30 min)

- Connexion **directe** à PostgreSQL Windows (`10.14.219.2:5433`) via le module Node.js `pg`
- Exécution de `SELECT row_to_json(t) AS r FROM "{table}" t` pour les 4 tables
- Sérialisation JSON et upload direct dans MinIO Bronze (`raw/database/{table}/latest/`)
- Écriture en staging `/staging/database/{table}.json`
- **Chargement complet à chaque exécution** (base statique, pas de CDC sur ce workflow)
- Planification : **toutes les 30 minutes**

WF-04 — Enrichissement Google Distance Matrix (CDC)

- Lit `salesshipmentheader.json` depuis le staging
- Enrichit avec les coordonnées des magasins d'origine (données statiques)
- Déduplique les paires origine/destination pour éviter les appels redondants
- Appel HTTP vers Google Distance Matrix API (batchSize: 1)
- Agrège les résultats (distance_km, duration_min)
- CDC + upload MinIO + staging `/staging/enrichment/enrichment_distance_matrix.json`

WF-05 — Orchestrateur principal (07h00 Europe/Paris)

- Vérifie l'existence des fichiers sources
- Déclenche séquentiellement : WF-01 → WF-02 → WF-03 → WF-04
- Vérifie les résultats de chaque sous-workflow
- Lance Apache Hop via une requête HTTP POST sur `http://172.18.0.6:9999/run-etl`
- Déclenche le workflow Gold (`main_etl_gold.hwf`)

Planification globale n8n :

Heure	Workflow	Action
06h00	WF-01	Ingestion CSV
06h05	WF-02	Ingestion JSON

Heure	Workflow	Action
06h30+	WF-03	Polling PostgreSQL (30 min)
07h00	WF-05	Orchestrateur → déclenche tout + Apache Hop
08h00	WF-04	Enrichissement Distance Matrix

6.3 Efficacité de l'automatisation ETL

Métrique	Valeur
Fréquence d'ingestion PostgreSQL	Toutes les 30 minutes
Fréquence d'ingestion fichiers plats	Quotidienne (06h00)
Détection de changement	Temps réel via CDC MD5
Volume évité (données inchangées)	100% — aucun rechargement si hash identique
Durée transformation Silver + Gold	~15-25 min (Apache Hop, volumes courants)
Déclenchement automatique	Oui — sans intervention humaine

7. Couche Silver — Nettoyage avec Apache Hop

Apache Hop prend en charge les transformations des données brutes issues du staging vers une couche intermédiaire Silver nettoyée et standardisée.

6 pipelines Silver

Pipeline	Source	Destination
<code>01_silver_clients.hpl</code>	<code>/staging/database/clients.json</code>	PostgreSQL silver
<code>02_silver_articles.hpl</code>	<code>/staging/database/articles.json</code>	PostgreSQL silver
<code>03_silver_shipmentheader.hpl</code>	<code>/staging/database/salesshipmentheader.json</code>	PostgreSQL silver
<code>04_silver_shipmentlines.hpl</code>	<code>/staging/database/salesshipmentline.json</code>	PostgreSQL silver
<code>05_silver_distances.hpl</code>	<code>/staging/enrichment/enrichment_distance_matrix.json</code>	PostgreSQL silver
<code>06_silver_flat_files.hpl</code>	<code>/staging/flat_files/data_merged.csv/.json</code>	PostgreSQL silver

Chaque pipeline applique : - Lecture JSON depuis le staging - Mapping et renommage des champs selon la convention du DWH - Typage des colonnes (dates, nombres, textes) - Chargement en mode **InsertUpdate** (upsert) dans PostgreSQL

8. Couche Gold — Modélisation dimensionnelle

7 pipelines Gold

Pipeline	Type	Table cible
01_dim_date.hpl	Dimension	dim_date
02_dim_customers.hpl	Dimension	dim_customers
03_dim_articles.hpl	Dimension	dim_articles
04_dim_shipment_headers.hpl	Dimension	dim_shipment_headers
05_dim_distances.hpl	Dimension	dim_distances
06_fact_shipment_lines.hpl	Fait	fact_shipment_lines
07_fact_ecommerce_lines.hpl	Fait	fact_ecommerce_lines

Dimension Date (01_dim_date.hpl)

Génération via PostgreSQL `generate_series` de toutes les dates entre **2010-01-01** et **2030-12-31** (~7 665 lignes), avec colonnes dérivées : `year`, `quarter`, `month`, `month_name`, `week`, `day_of_month`, `day_of_week`, `day_name`, `is_weekend`.

9. Data Warehouse PostgreSQL — Schéma & Modèle en étoile

Base : `data_warehouse_gold`

Tables de dimensions

```
dim_date          — 7 665 lignes (2010–2030)
  date_id (PK), year, quarter, month, month_name, week,
  day_of_month, day_of_week, day_name, is_weekend

dim_customers
  customer_id (PK), name, country, post_code, location_code,
  currency_code, salesperson_code
```

```

dim_articles
  article_id (PK), description, unit_of_measure, type,
  item_category, unit_price, unit_cost, costing_method

dim_shipment_headers
  header_id (PK), customer_id (FK→dim_customers), order_no,
  shipment_date, posting_date, ship_to_city, ship_to_country,
  ship_to_post_code, location_code, shipment_method, salesperson_code

dim_distances
  id SERIAL (PK), order_id, origin, destination,
  distance_km, duration_min, extracted_at

```

Tables de faits

```

fact_shipment_lines
  id SERIAL (PK)
  document_no → FK dim_shipment_headers(header_id)
  article_id → FK dim_articles(article_id)
  shipment_date → FK dim_date(date_id)
  line_no, quantity, uom_code, gross_weight, net_weight,
  unit_volume, location_code

fact_ecommerce_lines
  id SERIAL (PK)
  invoice_no, stock_code, description, quantity,
  unit_price, invoice_date, customer_id, country,
  country_iso2, total_amount

```

Modèle en étoile

```

          dim_date
          |
dim_articles — fact_shipment_lines — dim_shipment_headers — dim_customers

dim_customers — fact_ecommerce_lines

          dim_distances

```

10. Gestion des utilisateurs et accès à la base de données

Trois profils utilisateurs ont été configurés sur le Data Warehouse PostgreSQL (`data_warehouse_gold`), chacun disposant d'un niveau d'accès adapté à son rôle.

10.1 Utilisateur **nyanu** — Administrateur ETL (propriétaire)

- **Rôle** : Propriétaire de la base de données, compte principal utilisé par Apache Hop pour l'écriture des données.
- **Droits** : Tous les droits (CREATE, ALTER, DROP, INSERT, UPDATE, DELETE, SELECT) sur toutes les tables et schémas.
- **Utilisation** : Connexion par les pipelines Apache Hop, création et modification du schéma, opérations de maintenance.

```
-- Connexion Apache Hop (project-config.json)
PG_USER=nyanu / PG_PASSWORD=nyanu
```

10.2 Utilisateur **admin** — Administrateur analytique

- **Rôle** : Accès complet en lecture/écriture sur l'ensemble du Data Warehouse, à des fins d'administration et d'audit.
- **Droits accordés** :
 - **SELECT**, **INSERT**, **UPDATE**, **DELETE** sur toutes les tables (dimensions + faits)
 - **USAGE** sur tous les schémas
- Accès à toutes les vues et séquences
- **Utilisation** : Administration de la base, vérification de la qualité des données, requêtes d'audit cross-tables.

```
CREATE USER admin WITH PASSWORD '*****';
GRANT CONNECT ON DATABASE data_warehouse_gold TO admin;
GRANT USAGE ON SCHEMA public TO admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO admin;
```

10.3 Utilisateur **vente** — Accès restreint (lecture seule)

- **Rôle** : Utilisateur destiné à l'équipe commerciale pour la consultation des données de vente uniquement.
- **Droits accordés** :
 - **SELECT** uniquement (lecture seule)
- Accès restreint aux tables pertinentes pour l'analyse commerciale :
 - **fact_ecommerce_lines** — transactions e-commerce
 - **fact_shipment_lines** — lignes d'expédition
 - **dim_customers** — clients
 - **dim_articles** — catalogue articles
 - **dim_date** — dimension temporelle
 - **dim_shipment_headers** — en-têtes d'expédition

- **Aucun accès** à `dim_distances` ni aux opérations d'écriture.

```
CREATE USER vente WITH PASSWORD '****';
GRANT CONNECT ON DATABASE data_warehouse_gold TO vente;
GRANT USAGE ON SCHEMA public TO vente;
GRANT SELECT ON fact_ecommerce_lines TO vente;
GRANT SELECT ON fact_shipment_lines TO vente;
GRANT SELECT ON dim_customers TO vente;
GRANT SELECT ON dim_articles TO vente;
GRANT SELECT ON dim_date TO vente;
GRANT SELECT ON dim_shipment_headers TO vente;
```

Récapitulatif des accès

Utilisateur	Rôle	SELECT	INSERT/ UPDATE/ DELETE	DDL (CREATE/ DROP)	Tables accessibles
<code>nyanu</code>	Propriétaire ETL	✓	✓	✓	Toutes
<code>admin</code>	Admin analytique	✓	✓	✗	Toutes
<code>vente</code>	Equipe commerciale	✓	✗	✗	fact_ecommerce_lines, fact_shipment_lines, dim_customers, dim_articles, dim_date, dim_shipment_headers

11. Orchestration et planification

Déclencheur HTTP (ETL Runner)

Un microservice Flask (`etl_runner/app.py`) expose un endpoint HTTP pour découpler n8n d'Apache Hop :

- `POST /run-etl` → exécute
`hop-run.sh --project ETL_Projet --file main_etl_gold.hwf --runconfig local`
- `GET /health` → vérification de l'état du service
- **Timeout** : 2 heures pour les transformations longues
- Retourne : `exitCode`, `stdout_tail`, `stderr_tail`

Script de démarrage Apache Hop (`hop_start.sh`)

Ce wrapper : 1. Démarre un serveur HTTP Java (`HopApiServer`) sur le port 9999 2. Expose les endpoints `/run-etl` et `/health` 3. Corrige la configuration par défaut (`hop-config.json`) pour pointer sur le projet `ETL_Projet` 4. Gère les signaux `SIGTERM` / `SIGINT` pour un arrêt propre

Workflow principal Apache Hop

```
main_etl.hwf
├─ main_etl_silver.hwf → 6 pipelines Silver (clients, articles, headers,
lines, distances, flat_files)
└─ main_etl_gold.hwf → 7 pipelines Gold (5 dimensions + 2 faits)
```

12. Enrichissement des données — Google Distance Matrix

Problématique

Les données d'expédition contiennent des champs d'origine et de destination mais pas de distances réelles. L'enrichissement par l'API Google Distance Matrix permet d'obtenir la distance et la durée de trajet pour chaque itinéraire.

Implémentation

1. Lecture de `salessshipmentheader.json` (via staging)
2. Jointure avec les coordonnées statiques des entrepôts (code localization → ville)
3. Déduplication des paires (`(origine, destination)`) pour optimiser les appels API
4. Appel séquentiel (batchSize: 1) à Google Distance Matrix API
5. Parse de la réponse JSON (status, distance.value, duration.value)
6. CDC par MD5 + upload MinIO + staging
7. Chargement dans `dim_distances` via Apache Hop

13. Business Intelligence — Metabase & Dashboards

13.1 Connexion au Data Warehouse

Metabase (v0.52.9) est déployé dans un conteneur Docker (`analytics_metabase` , port 3000) et connecté directement au Data Warehouse PostgreSQL Gold (`data_warehouse_gold`) via le réseau Docker interne.

Paramètres de connexion Metabase → PostgreSQL : - Hôte : `dwh_postgres`
(résolution DNS Docker interne) - Port : 5432 - Base : `data_warehouse_gold` -
Utilisateur : `admin` (accès lecture/écriture sur toutes les tables)

13.2 Dashboards créés

Trois tableaux de bord ont été créés dans Metabase, basés sur des questions SQL et des visualisations natives.

Dashboard 1 — Analyse des ventes e-commerce

Ce dashboard exploite la table `fact_ecommerce_lines` jointe à `dim_date` et `dim_customers`.

Visualisations incluses :

Indicateur	Type de graphique	Question SQL (résumé)
Chiffre d'affaires total	Scorecard	<code>SUM(total_amount)</code> sur <code>fact_ecommerce_lines</code>
CA par mois	Graphique linéaire	CA mensuel via jointure <code>dim_date</code>
Top 10 pays par CA	Graphique à barres	<code>GROUP BY country ORDER BY SUM(total_amount) DESC</code>
Répartition des quantités par article	Graphique circulaire	<code>GROUP BY stock_code, description</code>
Nombre de transactions par jour	Courbe temporelle	Count par <code>invoice_date</code>
Panier moyen par client	Tableaux	<code>AVG(total_amount) GROUP BY customer_id</code>

Filtres dynamiques : période (date de début / fin), pays, stock_code — permettant une exploration interactive des données.

Dashboard 2 — Suivi des expéditions

Ce dashboard exploite `fact_shipment_lines` jointe à `dim_shipment_headers`, `dim_articles`, `dim_date` et `dim_distances`.

Visualisations incluses :

Indicateur	Type de graphique	Question SQL (résumé)
Nombre total d'expéditions	Scorecard	<code>COUNT(DISTINCT header_id)</code>
Expéditions par mois	Graphique à barres	<code>GROUP BY month, year</code> via <code>dim_date</code>
Top destinations (pays)	Carte choroplèthe	<code>GROUP BY ship_to_country</code>
Volume total expédié (poids)	Scorecard	<code>SUM(gross_weight)</code>
Distance moyenne par expédition	Scorecard	<code>AVG(distance_km)</code> via <code>dim_distances</code>
Délai moyen de livraison (min)	Scorecard	<code>AVG(duration_min)</code> via <code>dim_distances</code>
Articles les plus expédiés	Tableau	<code>GROUP BY article_id, description ORDER BY SUM(quantity) DESC</code>

Filtres dynamiques : plage de dates, pays de destination, méthode d'expédition.

Dashboard 3 — Analyse clients & géographie

Ce dashboard exploite `dim_customers`, `fact_ecommerce_lines` et `dim_shipment_headers`.

Visualisations incluses :

Indicateur	Type de graphique	Question SQL (résumé)
Nombre total de clients	Scorecard	<code>COUNT(DISTINCT customer_id)</code>
Répartition clients par pays	Carte / Barres	<code>GROUP BY country</code>
Top 10 clients par CA	Tableau	<code>JOIN fact_ecommerce_lines ON customer_id</code>
Clients par devise	Graphique circulaire	<code>GROUP BY currency_code</code>
Vendeurs et leur portefeuille client	Tableau	<code>GROUP BY salesperson_code</code>

Filtres dynamiques : pays, devise, code vendeur.

13.3 Connexion avec l'utilisateur `vente`

L'utilisateur `vente` (lecture seule sur les tables commerciales) peut être configuré comme source de données secondaire dans Metabase pour limiter l'accès des analystes commerciaux aux seules données qui les concernent, sans exposer les tables d'infrastructure (`dim_distances` , etc.).

14. Infrastructure Docker

`docker-compose.yml` — Services principaux

```
services:
  minio:          image: minio/minio:latest           – Ports: 9000, 9001
  n8n:            image: docker.n8n.io/n8nio/n8n       – Port: 5678
  apache-hop:     image: apache/hop-web:latest        – Port: 8080
  postgres_gold:  image: postgres:16                 – Port: 5433→5432
  metabase:       image: metabase/metabase:v0.52.9    – Port: 3000
```

Variables d'environnement critiques n8n

```
N8N_ENCRYPTION_KEY: "nyanu_secret_key_99"
NODE_FUNCTION_ALLOW_BUILTIN: "fs,path,crypto,os,http,https,net"
NODE_FUNCTION_ALLOW_EXTERNAL: "pg"
GENERIC_TIMEZONE: "Europe/Paris"
```

Ces variables sont critiques : elles permettent aux Code Nodes n8n d'accéder au système de fichiers (`fs`), au module de hachage (`crypto`) et au client PostgreSQL (`pg`).

15. Connectivité réseau Windows ↔ Linux

Approche retenue : connexion directe PostgreSQL

Les conteneurs Docker du stack ETL (réseau `172.18.0.0/16`) accèdent directement à la base PostgreSQL Windows via l'adresse IP `10.14.219.2` sur le port `5433` . La résolution de cette connectivité repose sur l'accès réseau hôte disponible depuis les conteneurs.

n8n se connecte donc sans intermédiaire :

Container n8n (172.18.0.5) → 10.14.219.2:5433 (PostgreSQL Windows)

Le module Node.js `pg` gère la connexion avec un timeout de connexion de 10 secondes et un timeout de requête de 30 secondes pour absorber les éventuelles latences réseau.

Les fichiers plats (`data.csv`, `data.json`) sont quant à eux accessibles via le **bind mount** `./data_sources` monté directement dans le conteneur n8n — aucune connexion réseau nécessaire pour ces sources.

16. Bilan, résultats et guide de démonstration

16.1 Récapitulatif de ce qui a été réalisé

Composant	Statut	Détail
Infrastructure Docker	✓ Fait	5 services, réseau bridge dédié
Ingestion CSV (CDC)	✓ Fait	WF-01, MD5, MinIO Bronze, staging
Ingestion JSON (CDC)	✓ Fait	WF-02, même pattern
Ingestion PostgreSQL (CDC)	✓ Fait	WF-03, 4 tables, polling 30 min
Enrichissement Distance API	✓ Fait	WF-04, Google Distance Matrix
Orchestrateur n8n	✓ Fait	WF-05, schedule 07h00, séquentiel
Silver Layer (Apache Hop)	✓ Fait	6 pipelines, upsert PostgreSQL
Gold Layer (Apache Hop)	✓ Fait	7 pipelines, 5 dims + 2 faits
Schéma DWH (Star Schema)	✓ Fait	schema_gold.sql, FK, index
Gestion utilisateurs DB	✓ Fait	nyanu (ETL), admin, vente (read-only)
Trigger HTTP ETL Runner	✓ Fait	Flask, Docker exec, 2h timeout
Connectivité Windows-Linux	✓ Fait	Connexion directe <code>pg</code> → 10.14.219.2:5433
Dimension Date	✓ Fait	2010–2030, generate_series
Metabase connecté au DWH	✓ Fait	PostgreSQL Gold, port 5432
Dashboard ventes e-commerce	✓ Fait	CA, top pays, articles, clients
Dashboard expéditions	✓ Fait	volumes, destinations, distances
Dashboard clients	✓ Fait	géographie, top clients, vendeurs
Documentation	✓ Fait	DOCUMENTATION.md, guides, README

16.2 Données chargées dans le Data Warehouse

Table	Source	Volume estimé
<code>dim_date</code>	generate_series	7 665 lignes
<code>dim_customers</code>	PostgreSQL clients	~300 clients
<code>dim_articles</code>	PostgreSQL articles	~500 articles
<code>dim_shipment_headers</code>	PostgreSQL shipmentheader	~400 expéditions
<code>dim_distances</code>	Google Distance Matrix API	~300 itinéraires
<code>fact_shipment_lines</code>	PostgreSQL shipmentline	~1 000 lignes
<code>fact_ecommerce_lines</code>	data.csv + data.json (merged)	~500 000 lignes

16.3 Guide de reproduction et démonstration (bout en bout)

Ce guide permet de reproduire l'intégralité du pipeline ETL depuis zéro.

Étape 1 — Démarrage de l'infrastructure

```
cd /home/nyanu/Documents/ETL_Projet
docker compose up -d
# Attendre ~60s que tous les services soient démarrés
docker compose ps # vérifier: all "Up"
```

Étape 2 — Vérification des sources

- Vérifier que `./data_sources/data.csv` et `./data_sources/data.json` sont présents
- Vérifier que ngrok est actif sur la machine Windows (PostgreSQL accessible)

Étape 3 — Lancement du pipeline depuis n8n

1. Ouvrir `http://localhost:5678` (n8n)
2. Activer et lancer manuellement **WF-05 (Orchestrateur)**
3. Observer l'exécution séquentielle : WF-01 → WF-02 → WF-03 → WF-04 → Apache Hop

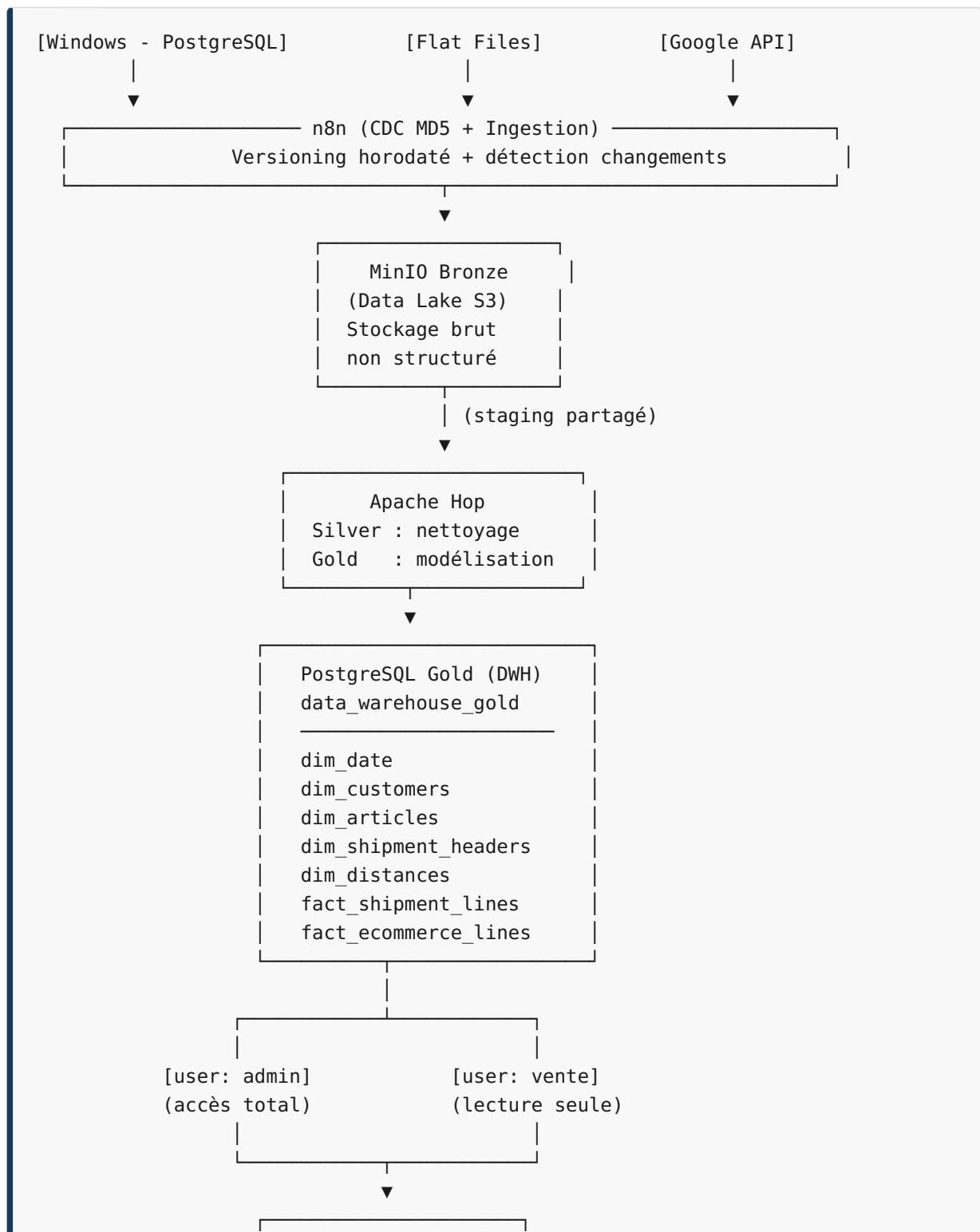
Étape 4 — Vérification du Data Warehouse

```
-- Connexion : psql -h localhost -p 5433 -U admin -d data_warehouse_gold
SELECT COUNT(*) FROM dim_date;           -- 7665
SELECT COUNT(*) FROM dim_customers;
SELECT COUNT(*) FROM fact_ecommerce_lines;
```

Étape 5 — Consultation des dashboards Metabase

1. Ouvrir `http://localhost:3000` (Metabase)
2. Naviguer vers **Dashboards**
3. Démontrer les 3 dashboards : Ventes, Expéditions, Clients
4. Utiliser les filtres dynamiques (dates, pays) pour illustrer l'exploration interactive

16.4 Architecture de données finale



Metabase
Dashboard Ventes
Dashboard Expéd.
Dashboard Clients

Rapport généré le 28 mai 2026